

1. GlideRecord – Grundlagen

GlideRecord wird verwendet, um Datensätze aus einer Tabelle zu lesen, zu filtern und zeilenweise zu verarbeiten.

```
var gr = new GlideRecord('table_name');
gr.addQuery('field_name', 'value');
gr.query();

while (gr.next()) {
    // gr.field_name enthält den Wert des aktuellen Datensatzes
    gs.info(gr.field_name.toString());
}
```

Häufige Methoden

Methode	Zweck
new GlideRecord('table_name')	Instanz für eine Tabelle erzeugen
.addQuery(field, value)	Filter hinzufügen
.addQuery(field, operator, value)	Filter mit explizitem Operator
.addNotNullQuery(field)	Filtert auf „Feld ist nicht leer“
.query()	Query ausführen
.next()	Zum nächsten Datensatz springen (in while -Schleife)
.get(sys_id)	Einzelnen Datensatz direkt per sys_id laden
.setValue(field, value)	Feldwert setzen
.update()	Änderungen speichern
.insert()	Neuen Datensatz anlegen
.getRowCount()	Anzahl gefundener Datensätze (nach .query())

Gültige Operatoren für addQuery

Operator	Bedeutung
=	gleich (Standard, kann weggelassen werden)
!=	ungleich
> , < , >= , <=	numerisch/Datum größer/kleiner
IN bzw. NOT IN	Wert in bzw. nicht in kommagetrennter Liste
CONTAINS	Text enthält Zeichenkette
ISEMPTY / ISNOTEMPTY	Feld leer/nicht leer

Zugriff auf Feldwerte

```

var value = gr.field_name.toString(); // String-Repräsentation, empfohlen für
Vergleiche
var raw   = gr.field_name;           // GlideElement-Objekt, kein direkter
String

```

Wichtig: Choice-Felder liefern über `.toString()` den **numerischen Code**, nicht das Anzeigelabel. Referenzfelder liefern über `.toString()` die `sys_id` des referenzierten Datensatzes.

Zugriff auf referenzierte Felder (Dot-Walking)

```

var referencedValue = gr.reference_field.some_field.toString();

```

2. GlideAggregate – Aggregationen

GlideAggregate wird verwendet, wenn nicht die einzelnen Datensätze benötigt werden, sondern aggregierte Werte (Anzahl, Summe, Gruppierung).

```

var ga = new GlideAggregate('table_name');
ga.addQuery('field_name', 'value');
ga.addAggregate('COUNT');
ga.groupBy('other_field');
ga.query();

while (ga.next()) {
    var count = ga.getAggregate('COUNT');
    var groupValue = ga.getValue('other_field');
}

```

Häufige Methoden

Methode	Zweck
<code>new GlideAggregate('table_name')</code>	Instanz für eine Tabelle erzeugen
<code>.addAggregate('COUNT')</code>	Anzahl der Datensätze zählen
<code>.addAggregate('SUM', field)</code>	Summe eines numerischen Feldes
<code>.addAggregate('MIN'/'MAX'/'AVG', field)</code>	Min/Max/Durchschnitt eines Feldes
<code>.groupBy(field)</code>	Gruppierung nach Feldwert
<code>.query()</code>	Aggregation ausführen
<code>.next()</code>	Zur nächsten Gruppe springen
<code>.getAggregate('COUNT')</code>	Aggregierten Wert der aktuellen Gruppe lesen
<code>.getValue(field)</code>	Wert des Gruppierungsfeldes lesen

Hinweis: GlideAggregate liefert nur aggregierte Werte, keine vollständigen Datensätze. Wenn zusätzlich einzelne Feldwerte eines konkreten Datensatzes benötigt werden, ist dafür GlideRecord das richtige Werkzeug.

3. Allgemeine Hinweise

- Beide APIs sind server-seitig und laufen synchron in Business Rules.
- `while (gr.next())` bzw. `while (ga.next())` ist die Standardform zum Durchlaufen von Ergebnissen – ohne diese Schleife wird nur die Query vorbereitet, aber nichts gelesen.
- Bei Referenzfeldern liefert der direkte Zugriff (`gr.field`) ein Objekt, kein primitives JavaScript-Datentyp – für Vergleiche und Verkettungen ist `.toString()` empfehlenswert.
- Fehler in der Query (z. B. nicht existierendes Feld) führen in der Regel nicht zu einer Exception, sondern zu einem leeren Ergebnis – für robuste Fehlerbehandlung empfiehlt sich eine explizite Prüfung (z. B. `.getRowCount()` oder `.isValidRecord()`), nicht nur `try/catch`.